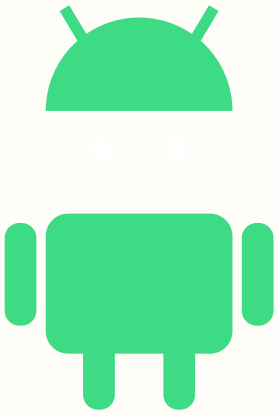
A brain-friendly guide for Android developers

Lifecycles, Signing & Publishing

From initState() to the Play Store: every lifecycle your Flutter app lives through, and how to sign, ship and update it.



what you know





what you'll learn

INSIDE:

- The 3 lifecycles: widget, app, and release
- State callbacks vs Activity callbacks
- AppLifecycleState, process death & rotation
- Versioning, build modes & flavors
- Keystores, upload keys & Play App Signing
- Publishing to Google Play (and App Store)
- After launch: crashes, updates, CI/CD

Lose your upload key and you'll have a very bad day. Chapter 5 makes sure you don't.

What's in this doc

1 **Three Lifecycles, One App**
Widget, app, and "idea to Play Store"

2 **The Widget (State) Lifecycle**
initState is your new onCreate

3 **The App Lifecycle**
Foreground, background, rotation and process death

4 **Building for Release**
Versions, build modes, flavors and app bundles

5 **Signing Your App**
Keystores, upload keys and Play App Signing

6 **Publishing to Google Play**
From Play Console to 100% rollout

7 **Life After Launch**
Crashes, updates, CI/CD - and the App Store

CHAPTER 1

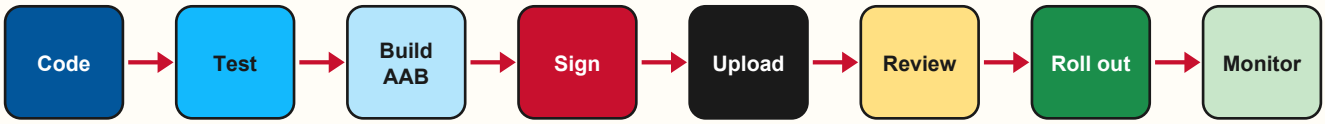
Three Lifecycles, One App

Widget, app, and "idea to Play Store"

When Android developers say "lifecycle", they usually mean the **Activity lifecycle**. In Flutter you'll deal with three different lifecycles. Keeping them apart in your head avoids a lot of confusion.

Lifecycle	What it describes	Android twin	Where
1. Widget (State) lifecycle	One StatefulWidget's State object is created, built many times, and disposed.	Fragment / View lifecycle	Chapter 2
2. App lifecycle	The whole app moves between foreground, background, hidden and detached.	Activity lifecycle / ProcessLifecycleOwner	Chapter 3
3. Release lifecycle	Code → test → build → sign → upload → review → roll out → monitor → update.	Exactly the same as Android!	Chapters 4-7

The release lifecycle at a glance



...then bump the version and go round again



RELAX

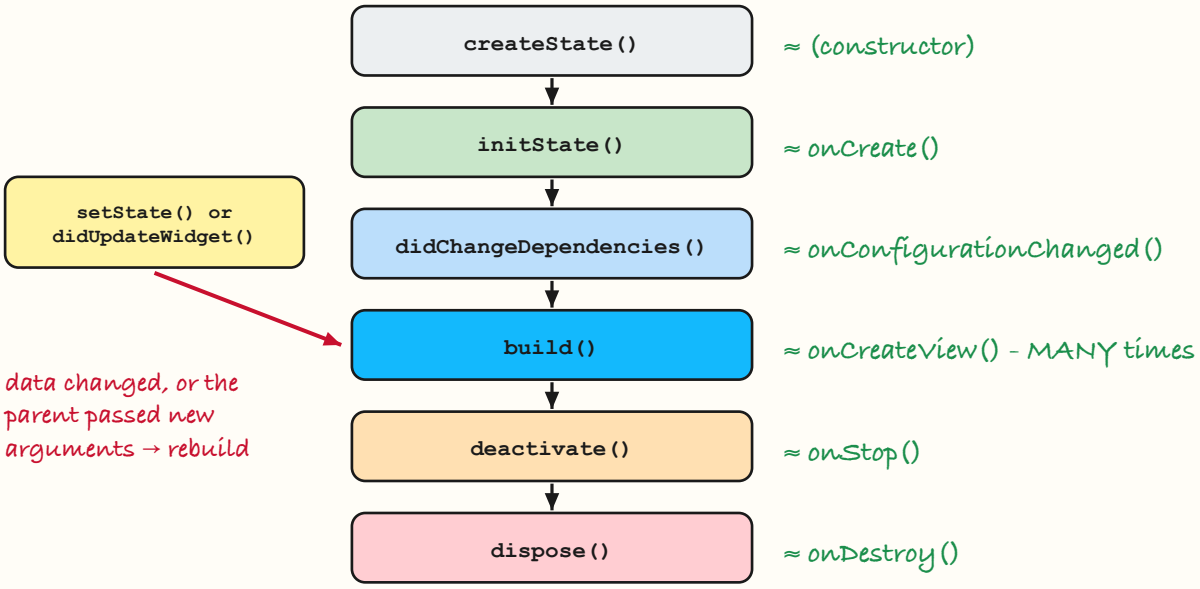
Good news for the third lifecycle: a Flutter Android release **is** an Android release. Same keystore, same AAB, same Play Console, same review. Your Android experience transfers 100%.

CHAPTER 2

The Widget (State) Lifecycle

initState is your new onCreate

Only **StatefulWidgets** have a lifecycle worth talking about - StatelessWidgets are just built and forgotten. The lifecycle belongs to the `State` object.



Method	When it runs	What to do there
`createState()`	Framework creates the State object.	Nothing. Never put logic here.
`initState()`	Once, when inserted into the tree.	Create controllers, subscribe to streams, start loading data. Call super first.
`didChangeDependencies()`	After initState, and whenever an InheritedWidget you read changes (Theme, MediaQuery, Provider).	Work that depends on context lookups.
`build()`	After any of the above, after setState, after didUpdateWidget.	Return widgets. No side effects, no I/O, keep it fast.
`didUpdateWidget(old)`	Parent rebuilt you with new constructor arguments.	Compare old vs widget; restart animations or reload if an id changed.
`deactivate()`	Removed from the tree (maybe re-inserted with a GlobalKey).	Rarely needed.
`dispose()`	Removed permanently.	Cancel timers & subscriptions, dispose controllers. Call super last.

A WELL-BEHAVED STATE

```
class _TickerState extends State<TickerWidget> {
  late final StreamSubscription<int> _sub;
  late final TextEditingController _ctrl;

  @override
  void initState() {                                          ← ≈ onCreate
    super.initState();                                        ← super FIRST
    _ctrl = TextEditingController();
    _sub = ticker.listen((v) => setState(() => _v = v));      ← subscribe once
  }

  @override
  void dispose() {                                            ← ≈ onDestroy
    _sub.cancel();                                            ← or it leaks & calls setState
    _ctrl.dispose();                                          ← controllers hold resources
    super.dispose();                                          ← super LAST
  }
}
```



Watch it!

"setState() called after dispose()" is the Flutter version of updating a destroyed Activity's views. It happens when an async callback finishes after the user left the screen. Fix: cancel subscriptions in dispose() and check `if (!mounted) return;` after every await.



RELAX

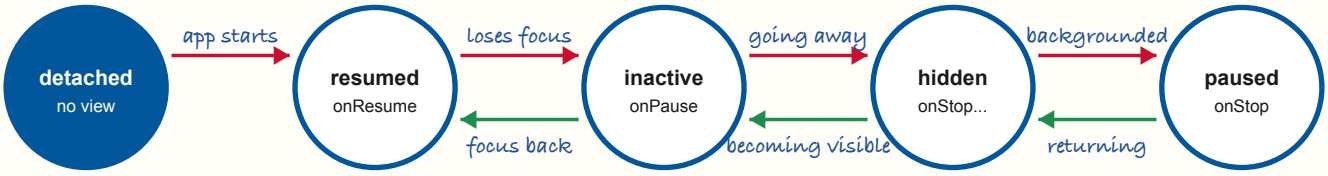
The app's **Components → Lifecycle** screen logs every one of these callbacks live. Insert/remove the child, pass it a new config, rotate the phone - and read the log. Ten minutes there beats an hour of reading.

CHAPTER 3

The App Lifecycle

Foreground, background, rotation and process death

The whole Flutter app lives inside one `FlutterActivity`, so the app lifecycle is basically that Activity's lifecycle, reported to Dart as `AppLifecycleState` values.



Going to background: resumed → inactive → hidden → paused. Coming back: the reverse. Flutter always walks through the in-between states.

State	Meaning	Android	Typical action
`resumed`	Visible and receiving input.	onResume()	Restart camera/sensors, refresh data, resume animations.
`inactive`	Visible but not focused (split screen partner, system dialog, incoming call overlay).	onPause()	Pause games/video. Good moment to hide sensitive content.
`hidden`	Not visible. Transitional state added in Flutter 3.13.	onStop() (starting)	Usually handled together with paused.
`paused`	In background, not visible, may be killed by the OS.	onStop()	Save drafts, release camera, stop location updates.
`detached`	Engine running without a view (starting up or Activity destroyed).	onDestroy()	Nothing reliable - don't count on it being called.

LISTENING TO APP LIFECYCLE (Flutter 3.13+)

```
late final AppLifecycleListener _listener;

@override
void initState() {
  super.initState();
  _listener = AppLifecycleListener(
    onResume: () => refresh(),
    onInactive: () => setState(() => _blur = true),
    onPause: () => saveDraft(),
    onStateChange: (s) => log('state: $s'),
  );
}

@override
void dispose() {
  _listener.dispose();
  super.dispose();
}
```

← ≈ ProcessLifecycleOwner observer

← hide secrets in Recents
← may be your LAST chance

Rotation: the good news

Look at the `<activity>` tag in `android/app/src/main/AndroidManifest.xml`. It declares `android:configChanges="orientation|screenSize|uiMode|locale|..."`. So **the Activity is NOT recreated on rotation**, dark-mode or language change. Flutter simply rebuilds the widgets that depend on `MediaQuery` / `Theme`. Your State objects and their fields survive. No more ViewModel-just-to-survive-rotation.

Process death: the bad news (same as Android)

When your app is in the background, Android may **kill the process** to reclaim memory. When the user comes back, Android recreates the Activity - and Flutter starts from `main()` again. All in-memory state is gone.

Strategy	Flutter tool	Notes
Persist important data as it changes	shared_preferences, sqflite/drift, files	The simplest, most robust fix.
Restore UI state (scroll position, tab, text)	RestorationMixin + RestorableInt etc., MaterialApp(restorationScopeId: 'app')	≈ onSaveInstanceState / SavedStateHandle
Restore the navigation stack	restorablePushNamed(...) / go_router restoration	≈ back stack restoration



SHARPEN YOUR PENCIL

Test process death yourself. Put the app in the background, then run the command below. Re-open the app from Recents. What survived?

```
adb shell am kill com.example.learnflutter.flutter_learning_app
```

there are no **Dumb Questions**

Q: **Is there an equivalent of onSaveInstanceState?**

A: Yes: the **restoration framework** (RestorationMixin). Opt in with a `restorationScopeId` on MaterialApp and restorable properties in your States. Many apps skip it and just persist data - which is fine.

Q: **What about Application.onCreate()?**

A: That's `main()`: runs once per process. Call `WidgetsFlutterBinding.ensureInitialized()` before using plugins, initialise Firebase, open databases, then `runApp()`.

Q: **Can I intercept the app closing or the back button on the root screen?**

A: Use `PopScope(canPop: false, onPopInvokedWithResult: ...)` to intercept Back (≈ OnBackPressedCallback), and `AppLifecycleListener(onExitRequested:)` on desktop.

CHAPTER 4

Building for Release

versions, build modes, flavors and app bundles

Versioning: one line to rule them all

pubspec.yaml

```
version: 1.4.2+37

# 1.4.2  -> build name   = versionName
# 37     -> build number = versionCode
```

← the only line you edit

← what users see in the store

← MUST increase on every upload

You can override it at build time - handy in CI: `flutter build appbundle --build-name=1.4.2 --build-number=37`



Watch it!

Play rejects an upload whose versionCode is not higher than every previous upload - even ones only on the internal track. Many teams derive the build number from the CI run number so it can never go backwards.

Build modes

Mode	Command	What you get
`debug`	`flutter run`	JIT, hot reload, asserts, debugger, debug banner. Slow-ish. Never ship it.
`profile`	`flutter run --profile`	AOT like release, plus DevTools performance tracing. Real device only.
`release`	`flutter build appbundle`	AOT native code, tree-shaken, no debugging. What you ship.

Flavors & environment config (≈ productFlavors + BuildConfig)

ANDROID (Kotlin)

```
// app/build.gradle.kts
flavorDimensions += "env"
productFlavors {
  create("dev") {
    dimension = "env"
    applicationIdSuffix = ".dev"
    buildConfigField("String",
      "API_URL", "\"https://dev.api\"")
  }
  create("prod") { dimension = "env" }
}
// code: BuildConfig.API_URL
```

FLUTTER (Dart)

```
// Same productFlavors block in android/app
// (Flutter uses it for the native side), then:

flutter run --flavor dev \
  --dart-define=API_URL=https://dev.api

flutter build appbundle --flavor prod \
  --dart-define-from-file=config/prod.json

// Dart code:
const apiUrl = String.fromEnvironment(
  'API_URL',
  defaultValue: 'https://api.example.com');
```



Watch it!

`--dart-define` values are compiled into the app binary. They are **configuration, not secrets**. Anyone can extract them from the APK. See Doc 4 (Security) for how to handle API keys.

The release commands

TERMINAL

```
# Google Play (recommended): Android App Bundle
flutter build appbundle --release --obfuscate --split-debug-info=build/symbols
#   -> build/app/outputs/bundle/release/app-release.aab

# APK for side-loading / other stores (one APK per CPU = smaller)
flutter build apk --release --split-per-abi --obfuscate --split-debug-info=build/symbols
#   -> build/app/outputs/flutter-apk/app-arm64-v8a-release.apk ...

# iOS
flutter build ipa --release --obfuscate --split-debug-info=build/symbols
```

there are no Dumb Questions

Q: **AAB or APK?**

A: Google Play requires **AAB** for new apps. Play then generates optimised APKs per device (only the right CPU architecture, screen density and language). Use APKs only for side-loading, testing, or stores that need them.

Q: **What is --split-debug-info and do I need to keep that folder?**

A: It moves Dart debug symbols out of the binary (smaller app, and needed for --obfuscate). **Keep the folder for every release** (archive it in CI): without it you can't turn obfuscated crash stack traces back into readable ones.

Q: **How big is a Flutter app?**

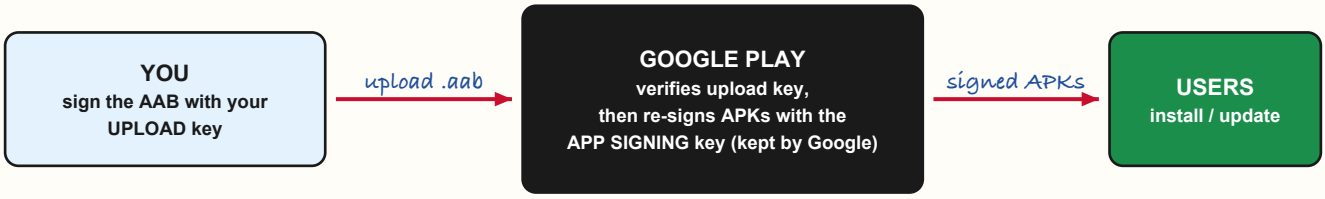
A: The engine + framework add a few MB (compressed download around 4-7 MB for a small app). Run `flutter build apk --analyze-size` to see exactly what takes space.

CHAPTER 5

Signing Your App

Keystores, upload keys and Play App Signing

Android refuses to install unsigned APKs, and the signature proves every update comes from the same developer. Flutter changes nothing here - it's all standard Android. But the modern Play flow uses **two keys**, and mixing them up is a classic mistake.



Upload key lost or leaked? Ask Play support to reset it - your app is safe.

App signing key lost? Can't happen with Play App Signing: Google keeps it.

Key	Who holds it	Purpose	If lost
Upload key	You (keystore .jks file)	Proves uploads come from you	Can be reset by Google Play support
App signing key	Google Play (secure HSM)	The key users' devices actually see	Must never change for the life of the app

Step 1 - Create an upload keystore (once, ever)

TERMINAL (macOS / Linux)

```
keytool -genkey -v \
  -keystore ~/upload-keystore.jks \
  -keyalg RSA -keysize 2048 \
  -validity 10000 \
  -alias upload
```

← ships with the JDK
← keep it OUTSIDE the project
← ~27 years
← remember this name

It asks for a keystore password, key password and your name/organisation. Put the passwords in a password manager right away.

Step 2 - Tell Gradle where it is (key.properties)

android/key.properties (git-ignored - NEVER commit)

```
storePassword=<keystore password>
keyPassword=<key password>
keyAlias=upload
storeFile=/Users/you/upload-keystore.jks
```

← same alias as in keytool
← absolute path

Step 3 - The signing config (already in this project!)

android/app/build.gradle.kts (simplified)

```
val keystoreProperties = Properties()
val keystorePropertiesFile = rootProject.file("key.properties")
if (keystorePropertiesFile.exists()) {
    keystoreProperties.load(FileInputStream(keystorePropertiesFile))
}
android {
  signingConfigs {
    if (keystorePropertiesFile.exists()) {
      create("release") {
        keyAlias = keystoreProperties["keyAlias"] as String
        keyPassword = keystoreProperties["keyPassword"] as String
        storeFile = file(keystoreProperties["storeFile"] as String)
        storePassword = keystoreProperties["storePassword"] as String
      }
    }
  }
  buildTypes {
    release {
      signingConfig = if (keystorePropertiesFile.exists())
        signingConfigs.getByName("release")
        else signingConfigs.getByName("debug")
      isMinifyEnabled = true
      isShrinkResources = true
    }
  }
}
```

← android/key.properties

← upload key if configured...

← ...debug key otherwise

← R8 for the Kotlin/Java side

Step 4 - Build and verify

TERMINAL

```
flutter build appbundle --release --obfuscate --split-debug-info=build/symbols

# Which certificate signed it? (SHA-256 should match your upload key)
keytool -printcert -jarfile build/app/outputs/bundle/release/app-release.aab
keytool -list -v -keystore ~/upload-keystore.jks -alias upload
```



Watch it!

If `key.properties` is missing, this project falls back to the **debug key** so learners can still run release builds. Play Console will reject a debug-signed bundle - make sure key.properties exists on the machine (or CI) that builds your real release.

KEYSTORE SAFETY CHECKLIST

- ☐ Keystore file stored outside the repo AND backed up (password manager / secure vault).
- ☐ key.properties, *.jks and *.keystore are in .gitignore (they are in this project).
- ☐ Passwords are stored in a password manager, not in a text file on your desktop.
- ☐ In CI, the keystore is a base64-encoded secret, decoded at build time.
- ☐ Play App Signing is enabled (it is by default for new apps).
- ☐ You recorded the SHA-1/SHA-256 fingerprints (needed for Google Sign-In, Firebase, Maps).

there are no Dumb Questions

Q: **Google Sign-In works in debug but fails in the Play Store build. Why?**

A: Because Play re-signs your app with the **app signing key**. Register that key's SHA-1 (Play Console → Setup → App signing) in Firebase / Google Cloud, in addition to your upload and debug keys.

Q: **What about iOS?**

A: Apple uses certificates + provisioning profiles managed in your Apple Developer account. Open `ios/Runner.xcworkspace` in Xcode, set your Team under Signing & Capabilities and enable "Automatically manage signing". Xcode does the rest.

CHAPTER 6

Publishing to Google Play

From Play Console to 100% rollout

Before you build: the pre-flight checklist

PRE-FLIGHT CHECKLIST

- ☐ **applicationId** is final (android/app/build.gradle.kts). It can never change after publishing.
- ☐ **App name**: android:label in AndroidManifest.xml.
- ☐ **Launcher icon**: generate with the flutter_launcher_icons package (adaptive icons too).
- ☐ **Splash screen**: launch_background.xml or the flutter_native_splash package.
- ☐ **Permissions**: only what you need; INTERNET in the main manifest if you use the network.
- ☐ **targetSdk** meets Google Play's current requirement (Google raises it every year - check Play Console).
- ☐ **Version** bumped in pubspec.yaml (build number higher than any previous upload).
- ☐ **Release build tested** on a real device: flutter run --release.
- ☐ **Obfuscation** on, symbols folder archived (see Doc 4).
- ☐ **Privacy policy** URL ready (required if you handle any personal data - most apps do).
- ☐ **Debug-only code** removed or guarded with kDebugMode.

Play Console, step by step

#	Step	Details
1	Create a developer account	One-time registration fee; identity verification required.
2	Create app	Name, default language, app or game, free or paid.
3	Set up your app (dashboard tasks)	Privacy policy, app access (test login for reviewers), ads, content rating questionnaire, target audience, news app, Data safety form, government app declarations.
4	Store listing	Short & full description, 512×512 icon, 1024×500 feature graphic, 2-8 phone screenshots (tablet if supported).
5	Internal testing track	Upload the .aab, add up to 100 testers by email. Available within minutes - no full review.
6	Closed testing	Bigger groups. New personal developer accounts must run a closed test with a minimum number of testers for a minimum period before production access (check the current rule in Play Console).
7	Production release	Create release → upload/promote AAB → release notes → review → start rollout.
8	Staged rollout	Start at 1-10% of users, watch crashes and ANRs in Android vitals, then increase to 100%. Halt if something is wrong.



promote the SAME tested bundle from track to track - don't rebuild

BRAIN POWER

Why is it safer to *promote* the exact bundle your testers used, rather than building a fresh one for production?

Data safety: what Flutter packages collect counts too

The Data safety form must describe everything **your app and its SDKs** collect - including Firebase Analytics, Crashlytics, ads SDKs and any plugin that sends data. Check each package's documentation for its data disclosure.

CHAPTER 7

Life After Launch

Crashes, updates, CI/CD - and the App Store

Watching for crashes

Tool	What you see	Package
Android vitals (Play Console)	Crash rate, ANR rate, slow startup, excessive wake-ups	Built in, free
Firebase Crashlytics	Dart + native crashes; FlutterError.onError & PlatformDispatcher.onError	firebase_crashlytics
Sentry	Crashes + performance tracing, self-hostable	sentry_flutter

CRASH REPORTING SETUP

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();
  FlutterError.onError =
      FirebaseCrashlytics.instance.recordFlutterFatalError;
  PlatformDispatcher.instance.onError = (error, stack) {
    FirebaseCrashlytics.instance
        .recordError(error, stack, fatal: true);
    return true;
  };
  runApp(const MyApp());
}
```

← errors inside the framework

← uncaught async errors

Reading obfuscated stack traces

TERMINAL

```
# Crash report shows "Warning: This VM has been configured to produce stack traces ..."
flutter symbolize -i crash_stack.txt -d build/symbols/app.android-arm64.symbols

# Crashlytics can do it for you if you upload the symbols after each build:
firebase crashlytics:symbols:upload --app=<FIREBASE_APP_ID> build/symbols
```

Shipping updates

BULLET POINTS

- Bump `version: x.y.z+N` - N must increase.
- Prompt users to update with Google's In-App Updates API via the `in_app_update` package (flexible or immediate).
- For "minimum supported version" gates, serve the minimum from your backend or Remote Config and block old builds politely.
- Remember: Dart code ships inside the app. Changing it always means a new store release (Google Play policy doesn't allow downloading new executable code).

Automate it: CI/CD

.github/workflows/release.yml (sketch)

```
name: android-release
on: { push: { tags: ["v*"] } }                          ← run when you push a tag
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: "17" }
      - uses: subosito/flutter-action@v2                ← installs Flutter
        with: { channel: stable }
      - run: flutter pub get && flutter analyze && flutter test   ← gate on quality
      - name: Restore keystore
        run: |
          echo "$KEYSTORE_B64" | base64 -d > $RUNNER_TEMP/upload.jks   ← from GitHub secrets
          cat > android/key.properties <<EOF
          storeFile=$RUNNER_TEMP/upload.jks
          storePassword=$STORE_PASSWORD
          keyPassword=$KEY_PASSWORD
          keyAlias=upload
          EOF
        env: { KEYSTORE_B64: "${{ secrets.KEYSTORE_B64 }}", ... }
      - run: flutter build appbundle --release --obfuscate
             --split-debug-info=build/symbols
             --build-number=${{ github.run_number }}    ← always increasing!
      - uses: actions/upload-artifact@v4                ← keep AAB + symbols
        with: { name: release, path: "build/app/outputs/bundle/release/
                  build/symbols" }
```

Next steps: upload straight to a Play track with **fastlane supply** or the *roadkll/upload-google-play* action. Codemagic and Bitrise offer Flutter-specific hosted pipelines.

And the App Store, in one page

	Google Play	Apple App Store
Developer account	Google Play Console (one-time fee)	Apple Developer Program (yearly fee)
Identity / signing	Upload keystore + Play App Signing	Certificates + provisioning profiles (Xcode can manage them)
Build	flutter build appbundle	flutter build ipa
Upload	Play Console / API	Xcode Organizer or Transporter app → App Store Connect
Beta testing	Internal / closed / open tracks	TestFlight
Manifest	AndroidManifest.xml	ios/Runner/Info.plist (usage descriptions for camera, location...)
Review	Usually hours to a few days	Usually 1-2 days; stricter UI/guideline checks
Rollout	Staged % rollout	Phased release over 7 days



SHARPEN YOUR PENCIL

Put these release steps in the right order (number them 1-7):

Step	Order
Promote to production at 10%	
Bump version to 1.1.0+12	
flutter build appbundle --obfuscate ...	
Upload to internal testing	
flutter test && flutter analyze	
Check Android vitals, go to 100%	
Archive build/symbols for this version	



BULLET POINTS

- State lifecycle: initState once → build many → dispose once.
- App lifecycle: resumed / inactive / hidden / paused / detached, via AppLifecycleListener.
- Rotation doesn't recreate FlutterActivity; process death still restarts from main().
- version: name+number in pubspec.yaml; the number must always increase.
- Upload key (yours, resettable) vs app signing key (Google's, permanent).
- Ship AABs, obfuscate, archive symbols, roll out in stages, watch vitals.

Exercise Solutions

Chapter 3 - process death

Everything that was only in memory is gone: the counter shown in widgets, scroll position, the navigation stack. What survived: anything persisted - in this app the Storage demo's counter and notes, and your theme choice (SharedPreferences).

Chapter 6 - why promote instead of rebuild?

A rebuild is a *different binary*: different dependency resolution, a different machine, maybe a different Flutter version. Promoting ships exactly the bits your testers validated.

Chapter 7 - release order

#	Step
1	Bump version to 1.1.0+12
2	flutter test && flutter analyze
3	flutter build appbundle --obfuscate ...
4	Archive build/symbols for this version
5	Upload to internal testing
6	Promote to production at 10%
7	Check Android vitals, go to 100%